CASE-STUDY PRACTICE | CAPSTONE 12 | STUDY EDITION

Customer-Support Agents: Supervisors, Validators and the Cost of a Wrong Block

Scenario questions on agents that resolve customer requests with checks around their actions, each opening into deeper follow-ups, with answers that explain how to think. Built on the AI System Design Primer case study of the same name.

Jump to the contents | The case in one page | Numbers to know | Decision table | Sources

HOW TO STUDY WITH THIS BOOK

1. Read the amber **concepts box** at the start of a part and look up anything unfamiliar. 2. Read a **scenario** and, before reading further, write down what you would ask and what you would check. 3. Read the green **how to think** box, which names the mental move the question tests. 4. Walk down the nested steps: cover each answer, predict it, then compare. Each answer exposes the next question. The last step of every question asks for the general lesson in one sentence.

The pattern behind most questions

- A number or a technique arrives with authority: a published benchmark, a cost estimate, a shard count, a rate.
- The interviewer asks what it assumes, and what happens when the assumption fails.
- A strong answer names the hidden assumption, puts a number on it, and says how to measure it on your own data.
- The last step usually reframes the original question.

EVIDENCE TAGS

source-confirmed: checked against the named primary source. **illustrative**: this document's planning assumption. **recomputed**: arithmetic re-derived from the stated inputs. **reasoned**: an argument from the design, not tested.

A NOTE ON SOURCES AND ON WHAT WAS CHECKED

The scenarios are written for this document; none comes from a real company question bank. Nothing here was run on a customer-support system: figures are the sources' or arithmetic from stated inputs, and error rates, catch rates, prices, latencies and handling costs are illustrative. The last page lists the sources and says how much of each was read. A separate Mock Interview edition (answers hidden) uses the same questions and numbering.

Contents

Every line below is a link: click it to jump to that page. The outline panel of your PDF viewer lists the same entries, and the “Contents” link in every page footer brings you back here.

Front matter

A The case in one page 3

B Numbers to know 4

C Decision table 5

Parts at a glance

1 Framing the problem: two cases, outcome against correctness, and the sizing 6

2 Data and consistency: customer memory, policy and a refund that happens twice 10

3 Agent design: supervisors, validators, bounded errors and what a check costs 14

4 Serving, evaluation and operations 18

Sources and confidence 22

Every question, by part

Part 1 Framing the problem: two cases, outcome against correctness, and the sizing 6

01 One system, or two cases? Sources 6

02 What does month one prove? Evaluation 7

03 Size the sessions Estimation 8

04 Check the 700 agents Estimation 9

Part 2 Data and consistency: customer memory, policy and a refund that happens twice 10

05 What does a unified customer record give an agent? Data 10

06 A refund and the very next message Consistency 11

07 The policy changed an hour ago Policy 12

08 Review the refund executor Code review 13

Part 3 Agent design: supervisors, validators, bounded errors and what a check costs 14

09 What do Sierra's supervisors do? Architecture 14

10 Why a separate check? Design 15

11 Does the validator pay for itself? Economics 16

12 Which instructions get which mechanism? Policy 17

Part 4 Serving, evaluation and operations 18

13 Check before the action, or after? Serving 18

14 The availability of a chain Reliability 19

15 Audit what the validator approved Monitoring 20

16 Risks, and ten times the volume Risk 21

The case in one page

A summary of the case study to read before the questions.

Problem	Resolve customer requests end to end, with an acceptable rate of wrong or policy-violating actions reaching the customer.
Two cases	Sierra: a platform with supervisors, task-based model selection and an Agent Data Platform. Klarna: its own assistant, powered by OpenAI per its release. No source links them.
Sierra's approach	Bounded error rates, not perfect adherence: supervisors filter input, audit every action and response, and can switch from observing to intercepting (source-confirmed).
Klarna's claims	2.3 million conversations in month one, two-thirds of chats; under 2 minutes against 11; satisfaction on par; 25% fewer repeat inquiries; 700 agents' work; 40 million dollars estimated (self-reported).
Later	Press coverage of 2025 interviews reports a cost focus that lowered quality and a return to hiring human agents (reported, secondary).
Not published	Error rates, catch rates, Klarna's architecture and Sierra's routing.
First to break	The cost of false alarms: wrongly blocked good actions become human escalations.
Easy-to-miss traps	Klarna is not shown to use Sierra. 700 agents and 11 minutes imply 2.9 agent-minutes per chat. A 112 times validator return becomes 1.7 once false alarms are counted.

Numbers to know

Each figure carries the tag that says how it was obtained. Klarna's figures are its own claims. Error rates, catch rates, prices and latencies are illustrative, because no source publishes them.

Quantity	Value	Evidence
Klarna's claims	2.3 million conversations, two-thirds of chats; 700 agents' work; under 2 minutes against 11; 25% fewer repeat inquiries; 23 markets; 35+ languages; 40 million dollars estimated 2024 profit improvement	source-confirmed (Klarna, self-reported)
Total volume	2,300,000 ÷ (2 ÷ 3) = 3.45 million chats a month; 115,000 a day; 76,667 AI-handled a day	recomputed
Sessions	0.89 a second × 120 s = at most 106; 2 to 4 times at peak	illustrative; recomputed
700 agents check	2.3 million × 11 min = 421,667 h = about 2,600 agents; 700 agents imply 2.9 agent-minutes per conversation	illustrative; recomputed
Hallucinations at scale	76,667 × 8 messages = 613,000 a day; one in ten thousand = about 61	illustrative; recomputed
Policy cache	3,194 conversations an hour; 1,278 actions; 5% affected = 64 actions under the old rule	illustrative; recomputed
Validator inputs	30,667 actions a day; 920 bad; 828 caught; 92 reach the customer; 0.0012 dollars a call = 36.80 dollars a day	illustrative; recomputed
Corrected economics	595 false escalations × 4 dollars = 2,380; net 1,723 dollars a day; ratio 1.7; precision of blocks 58%	illustrative; recomputed
Sensitivity	A bad action worth 2 dollars: net −761 dollars a day; at a 1% false-positive rate escalations cost about 1,190	illustrative; recomputed
Latency chain	400 + 900 + 500 = 1,800 ms against 900 ms	illustrative; recomputed
Availability	0.999^3 = 99.7003% (26.3 h a year); with active pairs 99.9997% (1.6 min a year); six replicas	recomputed
Audit sample	Bad share of approved actions 92 ÷ 29,244 = 0.31%; 138 bad found needs about 44,000 random audits	illustrative; recomputed
Ten times	Up to 1,060 sessions; validator 368 dollars a day; 5,950 false escalations = 23,800 dollars a day	illustrative; recomputed

Decision table

What this design chooses, and when to choose otherwise.

Decision	Default here	Choose otherwise when	Evidence
Check placement	Intercept before actions that move money; observe and audit for low-stakes answers	Observe only for reversible, low-cost answers	source-confirmed; reasoned
Validator independence	A separate call, a different context and, where possible, a different model	A self-check only for trivial actions	source-confirmed; reasoned
Tolerance	Strict and layered for adversarial input and sensitive topics; flexible for tone	Uniform strictness if every error costs the same	source-confirmed
Customer data	A unified record, with a strong read after an action	Separate tools if few systems are involved	source-confirmed; reasoned
Policy cache	Short lifetime or push invalidation for important rules	Long caching for rules that rarely change	reasoned
Refund execution	Atomic claim plus an idempotency key	A plain write only if duplicates are harmless	reasoned
Failover	Equivalent models evaluated per task	None if a single provider is acceptable	source-confirmed; reasoned
Audit	Stratified sample of approved actions	A random sample only if bad actions are common	reasoned

PART 1

Framing the problem: two cases, outcome against correctness, and the sizing

Keeping a vendor's design apart from a company's results, asking what a month-one metric proves, and checking a company's figures against each other.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Customer-service agent	A system that resolves a customer's request, such as a refund or a cancellation, by understanding it, consulting policy and customer data, and taking an action.
Resolution rate	The share of conversations handled to completion without a human. It does not say whether the actions were right.
Escalation rate	The share of conversations handed to a person, either because the agent could not complete the task or because a check blocked its action.
Outcome metric and correctness metric	An outcome metric measures speed, cost or satisfaction. A correctness metric measures whether the action was the right one. The first can look good while the second is poor.
Bounded error rate	Sierra's goal: not perfect adherence, but an error rate kept within a stated limit by layers of checks.
Little's law	Items in a system equal the arrival rate times the time each spends there.
FTE-equivalent	Full-time-equivalent: the number of full-time staff whose work the system does. It depends on how much time each case takes a person.
Self-reported figure	A number published by the company that benefits from it, usually shortly after launch. It is evidence, not an audit.
Where to read	Sierra: From LLMs to enterprise-grade agents (October 2, 2025) | Sierra: Agent OS 2.0, from answers to memory and action (November 5, 2025) | Sierra: Constellation of models (December 3, 2025) | Klarna press release, February 27, 2024

01 One system, or two cases?

SOURCES

SCENARIO

A colleague presents Sierra's supervisor architecture and Klarna's 2.3 million conversations as one deployment: Sierra's design, running at Klarna. Check what the sources say.

HOW TO THINK ABOUT IT

Read what Klarna's release says about who built the assistant, and what Sierra's posts say about Klarna.

Q What does Klarna's release say?

A That Klarna announced its AI assistant powered by OpenAI. The release names OpenAI and quotes OpenAI's COO. It does not mention Sierra (source-confirmed).

- The release gives results, a feature list and quotes, and no architecture.

Q What do Sierra's posts say about Klarna?

A The three posts read for this case study do not mention Klarna. They describe Sierra's own platform: models chosen by task, supervisors, an Agent Data Platform and failover.

- No source read here says Klarna is a Sierra customer.

Q So what can you say?

A Two cases. Sierra shows a documented platform design, with no published results or error rates. Klarna shows published results, with no published design.

- Putting them together would borrow Klarna's numbers for Sierra's design, or Sierra's design for Klarna's results.

Q What would you do in a presentation?

A Label each fact with its source, and mark anything that joins them as a teaching pattern, not as a fact about either company.

- The supervisor, specialist and validator design in this study is that kind of pattern.

Q Reframe it.

A Do not merge two sources into one system because they cover the same topic. Say who built what, and where the two overlap and where they do not.

Basis: Klarna's release and Sierra's posts are the sources (source-confirmed); the absence of a link is a statement about what the sources say.

02 What does month one prove?

EVALUATION

SCENARIO

Klarna's release says its assistant handled two-thirds of chats in its first month, resolved errands in under 2 minutes against 11, matched human satisfaction and cut repeat inquiries by 25%. A manager reads that as proof the system is correct. Press reports later describe a reversal. Say what the metrics show.

HOW TO THINK ABOUT IT

Sort each metric into outcome or correctness, then read what came later.

Q Which are outcome metrics?

A Two-thirds of chats handled, 700 agents' worth of work, resolution under 2 minutes, availability in 23 markets and 35 languages, and the \$40 million estimated profit improvement. They measure volume, speed and cost (source-confirmed).

- All are Klarna's claims, a month after launch.

Q Which is closest to correctness?

A The 25% drop in repeat inquiries, which Klarna ties to more accurate errand resolution. Satisfaction on par with humans is also a quality signal, and the release does not say how it was measured.

- A customer can be satisfied by a fast, friendly, wrong answer they cannot check.

Q What was reported later?

A Press coverage reports that in May 2025 Klarna's CEO told Bloomberg that the use of AI had been too focused on cost, which led to lower quality, and that Klarna would hire human agents again. A September 2025 Reuters interview is reported as saying the drive had gone too fast (reported, secondary).

- Klarna says it is combining scalable AI with high-quality human support.

Q What does that teach?

A A month-one outcome metric does not show long-run quality. Track correctness directly: audited samples of actions, repeat contact, reversals and complaints, over months.

- The reports are secondary accounts of interviews, so cite them as reported.

Q Reframe it.

A Ask whether a metric measures what the system produced or whether it was right. Optimize and report both, and keep checking after the launch glow.

Basis: The release's metrics are from Klarna (source-confirmed, self-reported); the later reversal is from press coverage of interviews (reported, secondary); the sorting into outcome and correctness is reasoned.

03 Size the sessions

ESTIMATION

SCENARIO

Klarna says 2.3 million conversations were two-thirds of its customer service chats in the first month, and that customers resolve errands in under 2 minutes. Estimate total volume and concurrent sessions, and say what the numbers are bounds on.

HOW TO THINK ABOUT IT

Divide to get the total, convert to a rate, then apply Little's law with the upper bound on time.

Q What is the total volume?

A 2,300,000 ÷ (2 ÷ 3) = 3,450,000 chats a month, so about 1,150,000 were handled elsewhere (source-confirmed inputs).

- A day is 115,000 chats, of which 76,667 were AI-handled.

Q What is the arrival rate?

A 76,667 ÷ 86,400 = 0.89 conversations a second.

- Klarna's release gives 3.4 million transactions a day for the company, so chats are about 3% of daily transactions.

Q How many sessions at once?

A 0.89 × 120 seconds = at most 106 sessions on average. Klarna says under 2 minutes, so 120 seconds is an upper bound on the time to resolve an errand.

- The time to resolve may differ from the length of a chat session.

Q What is the peak?

A An average is not a peak. A multiple of 2 to 4 (assumed) gives 210 to 420 sessions. The sources give no hourly profile.

- Provision for the peak, and keep headroom for failover.

Q Reframe it.

A Turn a monthly count into a rate and then into concurrency. Say which inputs are bounds, and size for the peak.

Basis: The figures are from Klarna's release (source-confirmed, self-reported); the multiple of 2 to 4 is illustrative; arithmetic recomputed.

04 Check the 700 agents

ESTIMATION

SCENARIO

Klarna says its assistant does the equivalent work of 700 full-time agents and that resolution took 11 minutes before. A reviewer asks whether those two figures are consistent with 2.3 million conversations.

HOW TO THINK ABOUT IT

Compute the agent-hours if every conversation took 11 minutes, then the minutes per conversation implied by 700 agents.

Q What if every conversation took 11 minutes of agent time?

A 2,300,000 × 11 minutes = 25.3 million minutes, 421,667 hours. At 160 hours a month per full-time agent (assumed), about 2,600 agents.

- That is almost four times Klarna's 700.

Q What does 700 imply?

A 700 × 160 hours × 60 = 6.72 million minutes. Divided by 2.3 million conversations, about 2.9 minutes of agent time per conversation.

- The hours per agent is an assumption, so the 2.9 shifts with it.

Q How can both be true?

A If the 11 minutes was elapsed time for the customer, including waiting, and agents handled several chats at once, each took far less agent time than 11 minutes.

- The release does not define either figure.

Q What should you do with them?

A Not combine them without definitions. Quote each as Klarna's claim, and say what a plausible reading is.

- A claim that depends on an unstated definition is weak evidence for planning your own staffing.

Q Reframe it.

A Check headline figures against one another with a quick division. When they pull apart, a definition is missing.

Basis: The 2.3 million, 700 agents and 11 minutes are from Klarna's release (source-confirmed, self-reported); the 160 hours a month is illustrative; arithmetic recomputed; the reading is reasoned.

PART 2

Data and consistency: customer memory, policy and a refund that happens twice

What unifying customer data buys, how stale policy turns into wrong actions, and how a check-then-act race refunds twice.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Agent Data Platform (Sierra)	Sierra's memory and intelligence layer: it unifies unstructured conversations with structured data such as customer records, billing, inventory, policies and transactions.
Customer context	What is known about one customer: orders, subscription, past conversations. It changes when the agent acts.
Policy knowledge	Rules that apply to everyone, such as refund eligibility. It changes rarely, and a change must reach every agent.
Strong and eventual consistency	With strong consistency a read after a write sees it. With eventual consistency a read can briefly return the old value.
Check-then-act race	Two operations both check a condition and then both act, because neither saw the other's write. The result is a duplicate action.
Idempotency key	A unique key sent with an action so that repeating the request has the effect of doing it once.
Conditional write	A write that succeeds only if a stated condition still holds at the time of writing, so only one of two racing writers wins.
Stale policy	A cached rule that has changed at the source. Actions taken under it follow the old rule.
Where to read	Sierra: From LLMs to enterprise-grade agents (October 2, 2025) | Sierra: Agent OS 2.0, from answers to memory and action (November 5, 2025) | Sierra: Constellation of models (December 3, 2025) | Klarna press release, February 27, 2024

05 What does a unified customer record give an agent?

DATA

SCENARIO

A team proposes giving the agent access to each backend system separately, one tool per system. Sierra describes a unified layer. Compare, using Sierra's November post.

HOW TO THINK ABOUT IT

Say what Sierra's layer unifies, what an agent can then do, and what concentrating the data costs.

Q What does Sierra describe?

A The Agent Data Platform is the memory and intelligence layer of its Agent OS. It unifies every conversation, the unstructured data in calls, chats and emails, with structured data: customer data, billing systems, inventory, policies and transactions (source-confirmed).

- The stated aim is context and continuity, from one-and-done transactions to lasting relationships.

Q What can an agent do with it?

A Remember prior conversations and preferences, connect the dots across time, anticipate needs and recommend the next best action, according to the post. A customer is not asked again for what they said in another channel.

- These are Sierra's claims. The post gives no measurements.

Q What does separate tooling lose?

A The join. An agent calling a billing tool and a CRM tool separately has to reconcile them in its own reasoning, on every turn, and it cannot see the conversation history that lives in a third place.

- That is an argument from the design, not a result in the post.

Q What does unifying cost?

A Concentration. Billing and account data from many systems sits in one layer, which becomes the component most in need of tight access control and tenant isolation.

- A platform serving many companies must keep each company's data apart.

Q Reframe it.

A A unified view helps most when decisions need facts from several systems at once. Then ask what the unified store now has to protect.

Basis: The description of the platform and its aims is from Sierra's November post (source-confirmed); the comparison with separate tools and the cost are reasoned.

06 A refund and the very next message

CONSISTENCY

SCENARIO

A customer asks for a refund and the agent processes it. Seconds later the customer writes, did my refund go through? A lagging read could show the order as not yet refunded. Work out what can go wrong, and what to require.

HOW TO THINK ABOUT IT

Lay out a timeline with a write, a replication lag and a second turn.

Q What is the risk?

A With eventual consistency, the next turn's context may still show last_action empty. The agent may think the refund has not happened and propose it again, or tell the customer it has not.

- Either is wrong: a double refund, or a contradictory reply.

Q Show a timeline.

A t = 0: refund executed and written. Replication lag is 2 seconds (assumed). t = 1.5 s: the customer's follow-up arrives and reads a replica that still has the old value.

- Voice and chat customers often reply within seconds.

Q What should the design require?

A A strong read, from the primary or a read-your-writes guarantee, for the fields an action just touched. Other fields can be eventually consistent.

- The sources do not say how Sierra or Klarna do it.

Q What else protects it?

A An idempotency key on the refund itself, so a repeat request cannot refund twice even if a stale read lets the agent propose it again (Question 8).

- Defense in depth: the read, the key, and a validator that checks the action against the record.

Q Reframe it.

A Ask which reads must see the last write. The ones after an action on the same record need to, and a lag on anything else is fine.

Basis: The need for a strong read follows from the design (Lesson 3's consistency material); the 2 second lag and 1.5 second reply are illustrative; the sources do not describe it.

07 The policy changed an hour ago

POLICY

SCENARIO

Refund policy changes at 10:00. The agents read a cached copy of the policy that refreshes every hour. A manager says an hour's delay is harmless. Size it.

HOW TO THINK ABOUT IT

Count actions per hour, then the share that the changed rule affects.

Q How many actions in an hour?

A Using the Klarna-scale rate: 76,667 AI-handled conversations a day ÷ 24 = 3,194 an hour. With 40% proposing an action (illustrative), about 1,278 actions an hour.

- Averages hide the daytime peak, where it may be several times more.

Q How many does the change touch?

A If 5% of actions fall under the changed rule (assumed), 64 actions an hour are decided under the old rule.

- Each is a possibly wrong refund or a wrongly refused one.

Q Is that harmless?

A It depends on the direction. A tightened rule means 64 refunds a day that should be refused, which are costly to reverse. A loosened rule means 64 customers wrongly refused, which are escalations and complaints.

- Neither is zero.

Q What are the options?

A Shorter cache lifetimes for rules that change rarely and matter a lot, a push invalidation when policy changes, and a validator that reads the policy fresh instead of the specialist's copy.

- The cost of freshness is more reads per turn, and it should be weighed against the cost of 64 wrong actions an hour.

Q Reframe it.

A A cache lifetime is a number of wrong decisions waiting to happen. Compute it from the action rate and the share the change touches.

Basis: The rates are from Klarna's release (source-confirmed) and illustrative assumptions (40%, 5%); arithmetic recomputed.

08 Review the refund executor

CODE REVIEW

SCENARIO

A colleague writes the executor below. In production, some customers are refunded twice, mostly when they send two messages quickly or when a retry follows a timeout. Find the faults.

```
def execute_refund(order_id, amount):
    order = db.get(order_id)
    if order["last_action"] is None:
        payments.refund(order_id, amount)
        order["last_action"] = "refund"
        db.put(order_id, order)
```

HOW TO THINK ABOUT IT

Walk two concurrent turns through the function, then a crash between the refund and the write.

Q What is the race?

A Two turns read the order at the same time, both see last_action empty, and both call the refund. The check and the action are separate steps.

- This is a check-then-act race.

Q What about a crash?

A The refund succeeds and the process dies before the write. A retry reads last_action as empty and refunds again.

- The refund happens before the record of it.

Q What does the payment call lack?

A An idempotency key, so the payment service cannot tell a retry from a new request.

- With a key derived from the order and the action, a repeat has no further effect.

Q State the fix.

A Claim the action with a conditional write first (set last_action only if it is empty), then call the refund with an idempotency key built from the order id, and mark it complete afterward.

- Test with two threads and with a forced failure between the steps.

Q Reframe it.

A Anything that can be retried or run twice at once needs a claim and a key. Record the intent before you act, and make the action repeatable safely.

REFERENCE CODE

```
def execute_refund(order_id, amount):
    key = f"refund:{order_id}"                          # same key for every retry
    claimed = db.update_if(order_id, where={"last_action": None},
                           set={"last_action": "refund_pending"})   # atomic claim
    if not claimed:
        return "already_claimed"                        # a second turn or a retry
    payments.refund(order_id, amount, idempotency_key=key)  # safe to repeat
    db.update(order_id, set={"last_action": "refund_done"})
    return "refunded"
```

Basis: The faults are standard concurrency errors and the fix was checked by hand and not run; the sources do not describe an executor.

PART 3

Agent design: supervisors, validators, bounded errors and what a check costs

What Sierra's supervisors actually do, why a check should be separate, how to set tolerance by kind of instruction, and the cost of a validator that blocks good actions.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Supervisory agent (Sierra)	An agent that manages a production agent to keep it within policy. Sierra's roles include input filtering, output interception and detecting underspecified situations.
Observe and intercept modes	In observe mode a supervisor watches and logs. In intercept mode it can stop or change what the agent does before it reaches the customer.
Validator	A separate check, in its own call, of a proposed action against policy and correctness before it executes.
False positive and false negative	A false positive blocks a good action. A false negative lets a bad one through.
Catch rate (recall)	The share of actually bad actions the validator blocks.
Precision of blocks	The share of blocked actions that really were bad. A validator with high recall and low precision floods people with escalations.
Tolerance by instruction type	Sierra's idea that an adversarial input needs maximum adherence and a phrasing choice needs little, so each gets a different mechanism.
Correlated failure	A shared blind spot: when two checks use the same model family, one miss can be a miss by both.
Where to read	Sierra: From LLMs to enterprise-grade agents (October 2, 2025) | Sierra: Agent OS 2.0, from answers to memory and action (November 5, 2025) | Sierra: Constellation of models (December 3, 2025) | Klarna press release, February 27, 2024

09 What do Sierra's supervisors do?

ARCHITECTURE

SCENARIO

A teammate draws Sierra's design as a router that sends each request to a domain specialist, with a validator behind it. Compare the drawing with what Sierra's posts say.

HOW TO THINK ABOUT IT

List the supervisor roles in the October post, then look at how the December post describes the constellation.

Q What are the supervisors?

A Every production agent is managed by several supervisory agents that play different roles and use different LLMs depending on the task. Each has a well-defined role, an individually controlled level of agency, and can be evaluated and improved independently (source-confirmed).

- The post calls them Jiminy Crickets.

Q What do they do?

A One filters input, detecting and intercepting threats from context poisoning to jailbreaks. Others audit an agent's every action and response for policy violations or harmful language, and for high-risk topics can switch from observing to intercepting. Others look for ambiguous situations.

- If a fix would materially change the meaning or add too much latency, a supervisor can escalate or end the conversation.

Q What is the constellation?

A Models chosen by kind of task: low-latency tool calling, high-precision classification, long-context reasoning and tone. The platform has modular task abstractions: retrieval, classification, tools, policies and tone (source-confirmed).

- It is a split by capability, not by billing or returns.

Q How does the drawing differ?

A Routing to a domain specialist is not described. The supervisors enforce guardrails. A router with domain specialists and a separate validator is a reasonable pattern to teach, and it is this document's design.

- Say so when you present it.

Q Reframe it.

A Draw what the source says, label your own additions, and do not retrofit a familiar pattern onto a vendor's description.

Basis: The supervisor roles and the constellation are from Sierra's October and December posts (source-confirmed); the point about routing is a reading of the posts.

10 Why a separate check?

DESIGN

SCENARIO

A specialist agent can be told, check your answer before sending. A reviewer asks why you pay for a second call by a separate validator. Argue it, and say what the second call cannot do.

HOW TO THINK ABOUT IT

Say what the checker shares with the author, and what a separate call changes.

Q What does a self-check share?

A The same context, training and framing that produced the answer. If a prompt framing led to a wrong refund amount, the same framing will usually approve it.

- Asking a model to review itself is the reflection pattern, and it is not independent.

Q What does a separate call change?

A It gets its own narrower context and one job: check the proposed action against the policy and the customer record. It does not inherit the author's reasoning.

- Sierra's supervisors are designed so each can be evaluated and improved independently, which supports separation.

Q What can it not do?

A Remove correlated failure. If the validator is the same model family, a shared blind spot can pass the same error. Sierra says different supervisors use different LLMs depending on the task, which helps.

- Use a different family or a rule-based check for the highest-risk actions, and make the verdict cite the policy it applied.

Q What does it cost?

A Another model call and its latency, a second retrieval of policy and context, and a service to keep available.

- Question 13 and Question 14 price those.

Q Reframe it.

A Independence comes from a different context and, where possible, a different model. A second pass by the same author is mostly a repeat.

Basis: Sierra's statements about independent supervisors and different LLMs are from the October post (source-confirmed); the argument about correlated failure is reasoned.

11 Does the validator pay for itself?

ECONOMICS

SCENARIO

An analyst says: the validator costs 37 dollars a day and prevents 4,140 dollars of bad actions, a 112 times return. Recompute it with the false alarms the analyst left out. All rates are assumptions.

HOW TO THINK ABOUT IT

Count the good actions that get blocked, price each escalation, then recompute the net and the precision.

Q What are the inputs?

A 76,667 conversations a day, 40% proposing an action: 30,667 actions. A 3% bad rate: 920 bad. A 90% catch rate: 828 caught, 92 reach the customer.

- A bad action is worth 5 dollars to prevent, and a validation call costs 0.0012 dollars (300 input and 20 output tokens at 3 and 15 dollars per million).

Q What is the analyst's number?

A Compute: 30,667 × 0.0012 = 36.80 dollars. Benefit: 828 × 5 = 4,140 dollars. Ratio 112.

- It counts only the model call.

Q What did it leave out?

A The good actions that the validator blocks. If it wrongly blocks 2% of the 29,747 good actions, that is 595 escalations a day. At an assumed 4 dollars of human handling each: 2,380 dollars.

- A human has to look at each blocked action.

Q What is the corrected result?

A 4,140 − 37 − 2,380 = 1,723 dollars a day, a ratio of 4,140 ÷ 2,417 = 1.7. The precision of blocks is 828 ÷ (828 + 595) = 58%.

- It is still positive and nowhere near 112.

Q How sensitive is it?

A If a bad action is worth 2 dollars, not 5, the benefit is 1,656 and the net is 1,656 − 37 − 2,380 = −761: negative. At a 1% false-positive rate the escalation cost is about 1,190 dollars.

- The false-positive rate and the cost of an escalation matter far more than the model call.

Q Reframe it.

A Count the cost of every wrong outcome, not only the one the design targets. A check that blocks good work has a bill, and it is usually the larger one.

Basis: All rates and prices are illustrative; arithmetic recomputed; the sources publish no error or catch rates.

12 Which instructions get which mechanism?

POLICY

SCENARIO

Sierra's October post sorts instructions by tolerance. Your team has four rules: refuse a prompt injection, never reveal another customer's data, collect an order number before a return, and be warm in tone. Assign a mechanism to each, and compute why the strict ones need layers.

HOW TO THINK ABOUT IT

Match each rule to a tolerance bucket, then compute how often a small error rate shows up.

Q What are the buckets?

A Adversarial input, sensitive topics and illegal activities: lowest tolerance, maximum adherence, and robotic replies are fine. Brand guidelines and sensitive policies: lower tolerance, high adherence with nuance. Standard procedures: medium tolerance, adhere to the spirit. Phrasing and tone: medium to high tolerance (source-confirmed).

- The post says an agent should have much less autonomy on sensitive topics than on ordinary questions.

Q Assign the four rules.

A Prompt injection and another customer's data: the first bucket, with an input filter and an output interceptor that can block. The order number: the third bucket, handled in the prompt and flow. Tone: the fourth, judged in reviews after the conversation.

- The first bucket can overcorrect, as the post says, and that is acceptable.

Q Why layers?

A The post says that at scale a one-in-ten-thousand hallucination rate is a daily occurrence. With 76,667 conversations a day and an assumed 8 agent messages each, that is about 613,000 messages and about 61 problem messages a day.

- A single mechanism with a small error rate still produces events every day.

Q What does bounded mean?

A The goal is an error rate under a stated limit, with layers that catch what slips past others, and not a promise of perfection.

- Set the limit per bucket, much lower for the first.

Q Reframe it.

A Sort rules by what a miss costs, and give the costly ones layers and a blocking mode. Let the cheap ones flex.

Basis: The buckets, the autonomy remark and the one-in-ten-thousand statement are from Sierra's October post (source-confirmed); the 8 messages a conversation is illustrative; arithmetic recomputed.

PART 4

Serving, evaluation and operations

Checking before or after acting, availability of a chain, auditing a validator, and what grows at ten times the volume.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Synchronous and asynchronous check	A synchronous check runs before the action and delays it. An asynchronous check runs after, and a bad action may already have happened.
Failover	Switching to an equivalent backup when a component degrades. Sierra's failover is across model providers.
Series availability	A chain of services is available only when all are. The availability is the product of the individual ones.
Stratified audit	Sampling items from chosen groups, such as near the validator's threshold, instead of at random, so rare errors are seen.
Confidence interval for a rate	The range in which a measured rate probably lies. Its width falls with the square root of the sample.
Repeat inquiries	Customers who come back about the same problem. Klarna reports a 25% drop and ties it to more accurate resolution.
Human-in-the-loop escalation	Handing a conversation to a person, with the context the agent gathered.
Drift	A slow change in the situation, such as new products or policies, that makes earlier behavior less correct.
Where to read	Sierra: From LLMs to enterprise-grade agents (October 2, 2025) | Sierra: Agent OS 2.0, from answers to memory and action (November 5, 2025) | Sierra: Constellation of models (December 3, 2025) | Klarna press release, February 27, 2024

13 Check before the action, or after?

SERVING

SCENARIO

A product owner wants the validator to run after the refund is sent, to save its latency. Sierra's supervisors have an observe mode and an intercept mode. Decide, and show the latency and the risk.

HOW TO THINK ABOUT IT

Add up the calls in each design, then say what an unchecked action can already have done.

Q What does the check cost in time?

A With assumed latencies of 400 ms to classify, 900 ms to draft and 500 ms to validate, in sequence: 1,800 ms against 900 ms for a single call, twice the floor.

- Streaming the reply hides part of it, as long as no unvalidated action text is released.

Q What does an after-the-fact check do?

A It removes the 500 ms from the action turn. The refund is already sent when the audit disagrees, and a reversal costs money and trust.

- For money movement and cancellations, that window is the risk.

Q What does Sierra describe?

A Both modes. Supervisors can switch from observe to intercept for high-risk topics, and a supervisor can escalate or end the conversation if a fix would add too much latency (source-confirmed).

- So the choice is per topic, not global.

Q What would you do?

A Intercept before refunds, disputes and cancellations. Observe and audit later for low-stakes, reversible answers such as order status.

- Review the split from audit results.

Q Reframe it.

A Spend latency where an action cannot be taken back, and save it where an error is cheap to fix.

Basis: The observe and intercept modes are from Sierra's October post (source-confirmed); the three latencies are illustrative; arithmetic recomputed.

14 The availability of a chain

RELIABILITY

SCENARIO

The design chains a supervisor, a specialist and a validator. Each is 99.9% available. A manager asks what the platform's availability is, and how Sierra's provider failover fits.

HOW TO THINK ABOUT IT

Multiply the three, then price redundancy.

Q What is the chain's availability?

A 0.999 × 0.999 × 0.999 = 0.997003, which is 99.7003%, about 26.3 hours of downtime a year.

- Worse than a single 99.9% service, which is about 8.8 hours.

Q What does redundancy do?

A An active pair per service is available unless both fail: 1 − 0.001^2 = 99.9999%. The chain then is 0.999999^3 = 99.9997%, about 1.6 minutes a year.

- The cost is six replicas instead of three, and the extra capacity must really be able to take over.

Q What does Sierra describe?

A Built-in redundancy across model providers for mission-critical tasks, with monitoring of latency, error rates and timeouts, and automated routing to healthier, equivalent models when a provider degrades (source-confirmed).

- That protects the model layer. It does not protect your own services in the chain.

Q What is the catch in failover?

A Prompts differ across model families, as Sierra's December post says, so a fallback model must be evaluated for each task before it is trusted with it. A failover that changes behavior is a new risk.

- Test the fallback on the task evaluations, not only on availability.

Q Reframe it.

A Multiply availabilities along the chain, and then say what redundancy buys at what cost. A backup you have not tested is not yet a backup.

Basis: The provider redundancy and the prompt difference are from Sierra's December post (source-confirmed); the availability figures are arithmetic (recomputed) on assumed 99.9% services.

15 Audit what the validator approved

MONITORING

SCENARIO

Nobody can read every approved action. The team wants an estimate of the validator's catch rate. Work out why a random audit does not give one and what to do instead.

HOW TO THINK ABOUT IT

Compute how many approved actions are bad, then how many random audits find enough of them.

Q How rare are the bad approved actions?

A The 92 bad actions that reach the customer are among about 29,244 approved ones (30,667 − 1,423 blocked): 92 ÷ 29,244 = 0.31%.

- The inputs are the Question 11 assumptions.

Q What does a random audit need?

A To estimate a catch rate near 90% within plus or minus 5 points, you need about 138 bad actions (1.96 × sqrt(0.9 × 0.1 ÷ n) = 0.05).

- At 0.31% of approved actions that means auditing about 138 ÷ 0.0031 = 44,000 of them.

Q What do you do instead?

A Sample from strata where bad actions concentrate: actions near the validator's threshold, large amounts, new customers, rarely seen request types, and actions where the specialist and validator scores disagree.

- A stratified sample finds errors much faster, and the estimate must be weighted back to the whole.

Q What else should you track?

A Escalation and resolution separately, error by domain, and repeat inquiries, which Klarna reports fell 25%. A rising escalation rate can mean more real problems caught or worse specialists.

- Sierra describes reviews after each turn and conversation, pairing fast high-recall detection with slower high-precision reasoning.

Q Reframe it.

A To measure a rare error, look where it lives. A random sample of a rare event is mostly the common case.

Basis: The post-turn review design is from Sierra's October post and the repeat-inquiry figure from Klarna's release (source-confirmed); the rates are illustrative; arithmetic recomputed.

16 Risks, and ten times the volume

RISK

SCENARIO

A launch review asks what can go wrong with a support agent platform and what grows at ten times Klarna's first-month volume. Use the sources and the earlier arithmetic.

HOW TO THINK ABOUT IT

List the risks the sources name or imply, then scale the quantities.

Q What are the risks?

A Errors that reach customers: hallucinations at a small rate that is a daily event at scale, prompt injection and context poisoning, and a policy gap (source-confirmed). Correlated failure of the checks, and quality decay after launch, which Klarna's later statements illustrate (reported).

- A cost focus can lower quality without any single alert.

Q What does ten times do to sessions?

A Up to 1,060 concurrent sessions on average, at most, and 2,100 to 4,200 at an assumed peak of 2 to 4 times.

- Provision for the peak and for failover.

Q What does it do to cost?

A Validator compute goes from 37 to 368 dollars a day. False escalations go from 595 to 5,950 a day at the same rate, 23,800 dollars at 4 dollars each.

- The human cost grows with the volume and dominates.

Q What does it do to audit?

A The same coverage needs ten times the sample. A stratified plan keeps it affordable.

- Automate triage of the strata.

Q Reframe it.

A Scale each cost with its driver and look at which is the largest. Here it is people handling false alarms, not model calls.

Basis: The risks are from the sources (source-confirmed) and press coverage (reported); the 10 times figures are illustrative arithmetic.

Sources and confidence

The case study this book is built on is the AI System Design Primer's Capstone #12. The sources are three Sierra blog posts of late 2025 and Klarna's own press release of February 2024, all read in full on 2026-10-17, plus press coverage of Klarna's later change of course, cited as reported. Klarna's assistant is described by its release as powered by OpenAI, and no source ties it to Sierra, so the two are treated as separate cases. Neither company publishes error rates or catch rates, and the supervisor, specialist and validator design in this book is a teaching pattern, not a documented architecture.

- Sierra: From LLMs to enterprise-grade agents, October 2, 2025, read in full 2026-10-17
https://sierra.ai/blog/enterprise-grade-agents
- Sierra: Agent OS 2.0, from answers to memory and action, November 5, 2025, read in full 2026-10-17
https://sierra.ai/blog/agent-os-2-0
- Sierra: Constellation of models, December 3, 2025, read in full 2026-10-17
https://sierra.ai/blog/constellation-of-models
- Klarna: Klarna AI assistant handles two-thirds of customer service chats in its first month, February 27, 2024, read in full 2026-10-17
https://www.klarna.com/international/press/klarna-ai-assistant-handles-two-thirds-of-customer-service-chats-in-its-first-month/